Title: UART Interrupt-Driven Flash Write Operations on STM32L431

Name: MD SHAKIL KHAN

Student ID: 2330130220

Date of Lab: November 10, 2025

Introduction-

Before this lab, it was known that Flash memory in microcontrollers is non-volatile, allowing data to persist after power loss, and that UART communication is commonly used for serial data transfer. The lab is about implementing interrupt-driven UART communication to enable non-blocking Flash memory operations, where incoming data is processed in the background while other tasks continue running. The purpose of this lab was to develop a system that collects text input via UART interrupts, stores it in Flash memory Sector 50, and verifies the write operation while maintaining continuous system operation through non-blocking architecture.

Materia

Apparatus	Expendables
• AHL-STM32L431 embedded development board	• USB cable
• PC with AHL-GEC-IDE v4.63	• None
• Serial terminal software (串口测试王 v3.1)	• None

Procedure-

The procedure was followed as outlined in the lab handout. The key steps involved:

1. Declaring global variables for UART buffer management and status flags.
2. Initializing UART communication at 115200 baud rate with receive interrupts enabled.
3. Modifying the UART interrupt service routine (UART_User_Handler) in isr.c to handle character echo, backspace processing, and input buffering.
4. Implementing Enter key detection to trigger Flash write operations.

5. Configuring Flash memory operations in main.c including sector erase, data writing using flash_write, and verification using flash_read_logic.
6. Testing the complete system using serial terminal software with various text inputs.
7. Verifying data persistence through power cycle tests.

Data-

Variables Used in the Project:

- uart_rx_buffer[128]: Buffer for storing received ASCII characters
- uart_rx_index: Index variable tracking current buffer position (0-127)
- uart_data_ready: Ready flag set to 1 when Enter is received
- Flash Sector: 50 (logical address starting at offset 0)

Observed System Behavior:

- Real-time character echo with backspace support functioned correctly
- LED continued blinking at ~1 second intervals during all UART operations
- Successful Flash writes verified for test strings: "Hello Nantong University 2025!" (31 bytes), "I AM THE BEST" (13 bytes), "GROK HELPS ME" (12 bytes)
- All read-back operations matched original input exactly
- Data persistence confirmed through power cycle tests (unplug/replug USB)

Calculations-

No complex numerical calculations were required. The primary measurement involved determining string length for Flash write operations using strlen((char*)uart_rx_buffer):

Example Calculation:

- Input string: "Hello Nantong University 2025!"
- String length: 31 characters = 31 bytes
- Flash operations: Erase Sector 50 + Write 31 bytes + Verify read-back

All Flash operations completed successfully without errors, confirming proper implementation.

Analysis

1. How do UART interrupts enable non-blocking system operation?
UART interrupts allow the system to handle incoming data in the background without stopping other tasks. When a character arrives, the interrupt service routine processes it immediately, then returns control to the main program. This ensures that time-critical operations like LED blinking continue uninterrupted while data reception occurs simultaneously.

2. Why is Flash memory sector erasure required before writing?
Flash memory technology requires that cells be reset to their erased state (typically all 1s) before new data can be written. Unlike RAM, Flash memory cannot overwrite existing data without first erasing the entire sector. This is a fundamental characteristic of floating-gate transistor technology used in Flash memory cells.

3. What measures prevent buffer overflow in this system?
The system implements multiple protections: the buffer size is fixed at 128 bytes, the index counter is limited to maximum 127, and null termination ensures string integrity. If the buffer approaches capacity, additional characters are rejected until space becomes available through processing.

Conclusions

The purpose of this lab was to create a UART interrupt-driven system for writing text data to Flash memory with verification capabilities. This was successfully accomplished; the system reliably receives text input, writes it to Flash Sector 50, verifies the write operation, and maintains data persistence through power cycles while allowing continuous operation of other system functions.

The experiment demonstrated excellent reliability with all test cases performing as expected. Potential sources of error include buffer overflow if input exceeds 127 characters, Flash memory wear from excessive write cycles, and serial communication errors from incorrect baud rate settings or electrical noise.

This project relates directly to real-world applications such as IoT devices storing configuration data, automotive systems saving calibration parameters, industrial equipment maintaining operation logs, and consumer electronics requiring non-volatile storage. The techniques learned are fundamental to developing efficient embedded systems that require both responsive user interfaces and reliable data storage capabilities.